

Differentially Private SQL Query Processing: An Empirical Evaluation of Systems and Their Trade-offs

Loïs Ecoffet ¹, Veronika Rehn-Sonigo¹, Jean-François Couchot¹, and Catuscia Palamidessi²

¹ Université Marie et Louis Pasteur, CNRS, institut FEMTO-ST (UMR 6174), F-25000 Besançon, France

² Inria Saclay, 1 rue Honoré d’Estienne d’Orves, Bâtiment Alan Turing, Campus de l’École Polytechnique, Palaiseau, France

{lois.ecoffet, veronika.sonigo, jean-francois.couchot}@umlp.fr and catuscia.palamidessi@polytechnique.edu

Abstract. Analytical SQL queries are essential for extracting insights from relational databases but concurrently introduce significant privacy risks by potentially exposing sensitive information. To mitigate these risks, numerous of Differentially Private mechanisms provide a solid theoretical foundation for producing statistical data that respects privacy. However, in order to integrate properly with existing Database Management Systems (DBMS), constraints relating to the relational nature of databases must be taken into account. This article presents a qualitative and quantitative analysis of the techniques used to process SQL queries, as well as DP-SQL systems – software that facilitates the implementation of data protection for relational databases. We first organize, through the lens of SQL’s relational constraints, the theoretical techniques proposed for privacy units, table-level privacy, joins, and aggregation. We then evaluate six DP-SQL systems on these constraints, using a benchmark built on the TPC-H schema. Our results reveal a trade-off: specialized systems offer predictable behavior over a restricted SQL fragment, while general-purpose systems broaden coverage at the cost of stability, exposing bugs, timeouts, or undetected privacy leaks.

Keywords: Differential Privacy, Relational Database, Empirical Evaluation, SQL

1 Introduction

Relational databases represent a foundational component of modern data management, facilitating the efficient storage, retrieval, and analysis of large-scale datasets. These databases are typically managed via a Database Management System (DBMS) utilizing Structured Query Language (SQL). Analytical queries, particularly those encompassing SPJA (Select-Project-Join-Aggregate) operations, are indispensable for extracting the actionable insights required for decision support and data science, and reporting.

Nevertheless, all query categories remain susceptible to privacy breaches. While the risk is manifest in queries returning raw, non-aggregated records—which directly expose individual data—vulnerabilities persist even within statistical queries. Specifically, prior research [?,?] demonstrates that sensitive individual information can be reconstructed from aggregated results.

To mitigate these risks, Differential Privacy (DP) [?] provides a formal guarantee that the inclusion or exclusion of a single record does not significantly alter the statistical output, a property known as *indistinguishability*. Currently established as the de facto standard for privacy preservation, DP shifts the focus from syntactic data properties—such as k -anonymity [?] or l -diversity, which are vulnerable to various linkage attacks—toward algorithmic constraints. It ensures that the mechanism remains robustly indistinguishable, independent of the underlying dataset or an adversary’s auxiliary knowledge.

To operationalize these principles, numerous frameworks have been proposed to facilitate DP-compliant query processing within relational DBMS [?,?,?,?]. These systems share the fundamental objective of enforcing DP by perturbing query outputs, thereby introducing an inherent trade-off between privacy guarantees and data utility. Hereafter, we refer to these systems as DP-SQL, that is to say, software that is implemented whilst taking into account the constraints associated with the relational nature of databases. They are further distinguished by several technical dimensions: (i) SQL compatibility, ranging from “out-of-the-box” support to specialized interfaces; (ii) privacy granularity (tuple-level vs. user-level); (iii) join sensitivity management; and (iv) software architecture (proxy-based vs. integrated). These DP-SQL systems are designed to be deployed by data practitioners who are not experts in DP. These systems must therefore cater to analysts’ use of SQL, and it is necessary to analyse the differences between the systems in relation to user requirements.

Evaluating these DP-SQL systems for practical deployment requires a rigorous utility comparison, as DP-compliant frameworks differ significantly in their noise injection strategies. The choice of sensitivity model and associated mechanisms directly influences the resulting error rates, statistical deviation, and computational overhead. While theoretical investigations provide a foundational understanding, they must be complemented by empirical evaluations to quantify performance metrics across diverse workloads.

Contributions This paper provides an analysis of the various issues raised by the use of SQL as a language for accessing databases protected by differential privacy (DP) mechanisms (Section ??). This examination of theoretical possibilities helps to understand the challenges and key points of analysis required to carry out a qualitative and quantitative analysis of DP-SQL systems (Section ??). The aim of this study is to enable data practitioners to choose a system based on the requirements for SQL support, usability and system efficiency.

All code used to produce the results is available as open source [?].

2 Related Works

Zhang et al. [?] present a comprehensive evaluation framework comparing five open-source differential privacy tools (OpenDP Smartnoise, Google DP, PyTorch Opacus, TensorFlow Privacy, and Diffprivlib) across both statistical query and machine learning tasks.

Zhu et al. [?] provide a comprehensive survey of differentially private data publishing and analysis, structuring the literature around two frameworks: mechanisms for

releasing query answers or synthetic datasets (transformation, partitioning, iteration, and query separation), and mechanisms for embedding privacy directly into analysis algorithms (Laplace/exponential and private learning frameworks). The survey compares existing approaches along query type, accuracy, and computational efficiency, and highlights open challenges such as handling high-dimensional data, reducing sample complexity, and extending differential privacy to coupled or correlated records. These benchmarks do not cover the full range of features specific to SQL.

Cebere et al. [?] are developing audit tools for privacy-preserving frameworks to verify that the theory of privacy protection is upheld in practice and that no bugs or implementation errors create loopholes in the theory. Their tools have revealed several bugs present in the frameworks.

Ngong et al. [?] present an analysis of human-machine interfaces, focusing on data practitioners (both experts and non-experts in DP) using 4 frameworks. The study measures learnability, efficiency, error prevention and user satisfaction in order to compare the tools, and concludes that a DP framework should provide “APIs that copy existing data science tools where possible, make privacy budget choices explicit, and raise errors when DP might be violated.”

Our work complements these studies; the addition of an SQL perspective enables new analyses, and the analysis of the systems’ usefulness goes beyond simple metrics (introduction of bugs, privacy violation and usability for data practitioners).

3 Background

This section introduces the fundamental concepts of DP. At the heart of DP lies indistinguishability (Section ??); this concept is closely linked to those of sensitivity and stability (Section ??), which enable the mechanisms to be calibrated to achieve this property. These three concepts are central to the majority of mechanisms and frameworks that utilise DP.

3.1 Differential Privacy

Let d denote a distance function quantifying the divergence between two databases. Two databases, D_1 and D_2 , are defined as neighbors if $d(D_1, D_2) = 1$.

Definition 1 (Differential Privacy (DP) [?]). A randomized mechanism $\mathcal{M}$ is (ϵ, δ) -differentially private $((\epsilon, \delta)$ -DP) if for any neighboring databases D_1, D_2 and a set of possible output S :

$$\Pr[\mathcal{M}(D_1) \in S] \leq e^\epsilon \times \Pr[\mathcal{M}(D_2) \in S] + \delta \quad (1)$$

Differential privacy establishes a formal framework for ensuring the indistinguishability of neighboring databases. This guarantee ensures that the presence or absence of a specific individual’s record cannot be reliably inferred by an observer. More precisely, the statistical distribution of query results remains nearly identical regardless of whether a single individual’s data is included in the dataset. Consequently, the choice of

the distance metric d directly governs the “unit of privacy”, thereby defining the scope of the protection guarantee (see Section ??).

Intuitively, the parameter ϵ quantifies the information leakage regarding an individual, a concept formally referred to as the “privacy budget.” A lower budget value corresponds to a higher degree of privacy preservation. Each DP query executed against a database consumes a portion of this allocated budget; consequently, queries must be managed judiciously to prevent budget exhaustion. The total budget may be partitioned to accommodate multiple requests or the sequential application of various mechanisms. This principle is known as *composition* and constitutes a non-trivial challenge in DP system design.

The parameter δ denotes the probability that the mechanism may fail to uphold the ϵ -indistinguishability guarantee, potentially resulting in information leakage exceeding the defined privacy budget. Maintaining this failure probability at a negligible level is critical for ensuring robust privacy protection [?]. The inclusion of this relaxation parameter facilitates the use of more flexible mechanisms—such as the Gaussian mechanism—that would be inadmissible under the stricter constraints of pure DP (where $\delta = 0$).

3.2 Sensitivity and Stability

A standard technique for ensuring that a mechanism adheres to the DP property involves the injection of calibrated noise, which is parameterized by the query sensitivity and the privacy budget ϵ . Sensitivity quantifies the maximum potential impact that a single record can exert on the query result—specifically, the maximum variation between outputs across any two neighboring databases—and is formalized as follows:

Definition 2 (Global Sensitivity (GS)). *The global sensitivity of a query Q , denoted as GS_Q , is defined for any pair of neighboring databases D_1 and D_2 as:*

$$GS_Q = \max_{D_1, D_2: d(D_1, D_2)=1} \|Q(D_1) - Q(D_2)\| \quad (2)$$

To address the theoretically unbounded sensitivity of summation-based queries, frameworks employ clamping. This technique transforms each input value v of a restricted range $[L, U]$ into a bounded value $v' = \max(L, \min(v, U))$. The global sensitivity of the SUM operator is thus reduced to $GS_{sum} = \max(|L|, |U|)$.

Notably, GS_Q may be unbounded for certain query types, necessitating the addition of infinite noise and rendering the resulting query response unusable for analytical purposes.

A more general alternative solution to the issue of unbounded GS is Local Sensitivity (LS), which characterizes the sensitivity of a query Q relative to a specific database instance D_1 .

Definition 3 (Local Sensitivity (LS)). *For a fixed database D_1 and its set of neighboring databases, the local sensitivity of a query Q at D_1 , denoted $LS_Q(D_1)$, is defined as:*

$$LS_Q(D_1) = \max_{D_2: d(D_1, D_2)=1} \|Q(D_1) - Q(D_2)\| \quad (3)$$

However, LS cannot be directly utilized for noise calibration in practice. Because the required noise magnitude becomes data-dependent, the noise level itself may inadvertently leak sensitive information about the underlying dataset [?].

This challenge can be mitigated by introducing Smooth Sensitivity (SS) [?], which serves as a smooth upper bound on LS. By ensuring the sensitivity measure itself changes slowly across neighboring databases, SS allows for noise calibration that does not inadvertently disclose sensitive information. However, computing SS_Q is NP-hard for numerous queries Q [?], particularly those involving JOIN operations. To alleviate this computational complexity, Elastic Sensitivity (ES) [?] was developed. Its value is derived from the query structure Q and the maximum number of matching records a single source record can produce in the resulting joined relation, a property which extends c -stability recalled hereafter. Notably, ES provides a smooth upper bound on SS which is computable in linear time.

Definition 4 (c -stability [?]). A transformation T is said to be c -stable if, for any two input databases D_1 and D_2 , the distance between their transformed outputs is bounded by a factor c of their initial distance:

$$d(T(D_1), T(D_2)) \leq c \cdot d(D_1, D_2) \quad (4)$$

A transformation with bounded stability propagates the DP guarantees provided at its output back to its inputs, scaled by the stability constant c . This property allows for the compositional analysis of complex queries by tracking how each operator affects the cumulative privacy loss.

Theorem 1 ([?]). If a transformation T is c -stable and a mechanism $\mathcal{M}$ is (ϵ, δ) -DP, then the composed mechanism $\mathcal{M} \circ T$ is $(c \cdot \epsilon, c \cdot e^{c-1} \delta)$ -DP [?].

4 DP-SQL Systems

Differentially private SQL systems have evolved from narrow support for a few statistical queries to broader engines that can execute richer relational workloads.

The use of SQL as a query language presents specific constraints and challenges that these systems must address in order to ensure privacy whilst producing usable statistical results. In this section, we will examine the challenges associated with the relational aspect of DBMSs and how these systems address these issues.

4.1 Privacy Unit

The definition of a privacy guarantee is fundamentally dependent on the *privacy unit* designated for protection. In relational databases, the representation of personal data dictates the configuration of the privacy model, primarily categorized as either *tuple-level* or *user-level* privacy.

Tuple-level Privacy: Under this model, personal information is assumed to be contained within a single record. Each individual is represented by a unique tuple; consequently, protecting this specific tuple is sufficient to preserve the individual's privacy.

This is the standard assumption in many early DP applications where databases are flattened or de-normalized.

User-level Privacy: In contrast, user-level privacy accounts for scenarios where an individual's data is dispersed across multiple rows or tables. Each record containing sensitive information is mapped to a Personal Identifier (PID). Wilson et al. [?] introduce a generic mechanism for bounding user contribution private aggregation. Amin et al. [?] propose a characterization of the trade-off between privacy and utility and show a "sweet spot" for choosing the bounds on contributions. Liu et al. [?,?] examine the impact of the user-level on specific tasks, whilst minimising the noise introduced by the mechanisms.

4.2 Table Privacy

Early interpretations of the DP assumed that all data in every table had the same level of privacy. Subsequently, more lenient interpretations of the relational model were proposed with the aim of establishing a hierarchy of privacy levels for each table. Dong and Yi [?] propose a formulation that utilises primary key/foreign key relationships to determine whether a table contains private attributes or not. This interpretation of a database schema enables systems to process the DP at user level [?,?], but also to implement optimisations on public tables by not treating this information as confidential [?,?].

This interpretation is refined by [?], which proposes a protocol-based model of privacy levels, and a method for transforming this high-level model into technical constraints on the database and its mechanisms. According to [?], it is possible to set the privacy level for each table, but also to designate certain attributes as public.

It is important to note that the level of privacy of tables depends not only on the data stored, but also on the constraints governing the addition and removal of data from the database. For example, a table containing information on nations is public according to both definitions [?,?], but if adding a person with a new nation triggers a cascade update to create an entry in the nations table, then the addition of that person becomes observable. When a table is considered public, it is therefore necessary to ensure that it is not updated based on other data. This logic must therefore be handled by a software layer.

4.3 SQL Queries

SQL queries can be summarised as the core SPJA (Select-Project-Join-Aggregate) construct. These introduce specific constraints and solutions relating to privacy.

Selection and Projection are the easiest operators to support because they do not, by themselves, create the kind of fan-out that makes sensitivity explode. In practice, they are usually absorbed into a larger DP plan: the system rewrites the query, tracks how the filter and column drop affect the downstream statistic, and spends the privacy budget on the released answer rather than on the relational filtering step itself. This is why most systems treat them as part of the query compilation layer rather than as separate privacy primitives.

Joins have an unbounded stability, as a tuple can be linked to an arbitrary number of other tuples. To resolve this issue, [?, ?, ?] limit the types of join that are possible. Johnson et al. [?] provide a first approach to handling general equijoins by measuring the maximum impact of the join on stability. Rastogi et al. [?] and Chen and Zhou [?] enable the handling of counting conjunctive queries (a subset of queries involving joins), and [?] introduces a notion of optimality to reduce the error introduced in these queries. DJoin [?] handles distributed joins using privacy-preserving set-intersection and combination primitives, which lets two parties answer join-style questions without revealing their raw tables to each other. Later work extends the same theme to unrestricted joins and continual observation, replacing worst-case bounds with instance-specific or empirical sensitivity so that the noise matches the actual join structure more closely [?].

Aggregations are the most mature part of the literature, but they are not a single problem. They can be partitioned into three cases with different behavior.

Counting queries and Histograms: [?] introduces the Laplace mechanism to protect queries with bounded sensitivity. This mechanism is optimal for counting queries [?]. However, the question of optimality is more complex when considering multiple consecutive counting queries (batch mode) [?] or a histogram.

Histograms deserve separate mention because they sit at the boundary between aggregation and query workloads. They are not just collections of independent counts; they are correlated releases whose utility depends on internal consistency. Strategy-query methods, consistency enforcement [?], or hierarchical [?] releases all aim to preserve that structure, which is why histogram mechanisms often give much better utility than independent per-bin noise [?, ?].

Sum-like queries The SUM operator occupies a central role in SQL workloads and serves as the basis for deriving other aggregations such as AVG, VAR, or STDDEV. In the context of differential privacy, answering sum queries over numerical attributes is a non-trivial problem, as the addition or removal of a single tuple can have a dramatic effect on the query result, leading to potentially unbounded sensitivity.

Wu et al. [?] propose a set of techniques for solving sums by imposing a bound on the sensitivity. To respond to a single query, it is possible to define bounds a priori (via a scheme defined independently of the data [?, ?, ?, ?]), or to use part of the budget to compute bounds using the private data. To do this, a threshold must be calculated; two approaches are proposed: using the recursive mechanism [?] or Sparse Vector Technique (SVT) [?]. However, using part of the budget degrades the results; ultimately, adding noise that can be arbitrarily large leads, in practice, to a greater loss of precision [?].

To respond to a workload of (a set of) sum queries, a generalisation of the Matrix Mechanism [?] is possible, with several strategies for computing the matrix.

Order statistics Order statistics occupy a distinctive position in the differential privacy literature, as their global sensitivity is determined by the domain range rather than by the magnitude of individual contributions. Early Laplace-based approaches suffered from poor utility for this reason; smooth sensitivity-based methods [?] improved on this by calibrating noise to the local behavior of the quantile function rather than its worst-case sensitivity. Exponential mechanism approaches can also be used, penalising ranks far from the desired quantile, though this remains problematic for endpoints (MIN

and MAX), where a single rank change can yield very different results. Durfee [?] addresses the unbounded-domain case with an iterative guess-and-check procedure built on AboveThreshold (SVT) [?], which is particularly well-suited to extreme quantiles. Chaves et al. [?] instead tackle the case of a known domain, showing that the exponential mechanism overestimates the noise needed and proposing the Smooth Noisy Max mechanism, which calibrates noise to the instance-specific smooth sensitivity via a report-noisy-max architecture.

4.4 Systems Architecture

We identified two logical architectures. These are defined in terms of the level of information the software requires in order to implement the mechanisms.

Definition 5 (Integrated Architecture). *A system with an integrated architecture requires full or partial access to sensitive data in the database.*

Consequently, a system with an integrated architecture must address issues relating to security, cryptography and privacy, as it handles just as much sensitive data as the database itself.

Definition 6 (Proxy Architecture). *A system with a proxy architecture requires access to statistics on the database data.*

A proxy system therefore processes the same sensitive information as that returned by a statistical database.

DP framework for SQL Some tools [?,?,?,?] do not allow for immediate SQL integration. They therefore offer a framework designed to facilitate data planning (automatic budget management, possible operations, stability management, etc.). In this study, we focused on two of these frameworks: PINQ because it is one of the first frameworks, and Tumult because it is based on OpenDP, a high-potential DP library. OpenDP plays an important role as shared infrastructure in the differential privacy community, rather than as a single tool. It aims to provide a set of differentially private building blocks that are reviewed and maintained by the community, instead of one closed, single-purpose system [?]. Its core contribution is a programming framework that tracks stability and sensitivity through the composition of elementary transformations and measurements, and in this sense it generalizes the compositional analysis recalled in Definition ?? and Theorem ?. It is designed to let algorithmic contributions from the research community be reviewed, certified, and reused rather than be re-implemented from scratch each time. OpenDP is central to the DP-SQL systems studied in this paper: two of the six systems we evaluate, SmartNoise SQL [?] and Tumult Analytics [?], are built directly on top of OpenDP’s core. To enable the integration of SQL with these frameworks, it is sufficient to parse the query and execute functions provided by the framework in the order in which the SQL constructs are applied. Figure ?? illustrates how to implement a DP-SQL system that provides minimal SQL support whilst incorporating the features of a DP framework

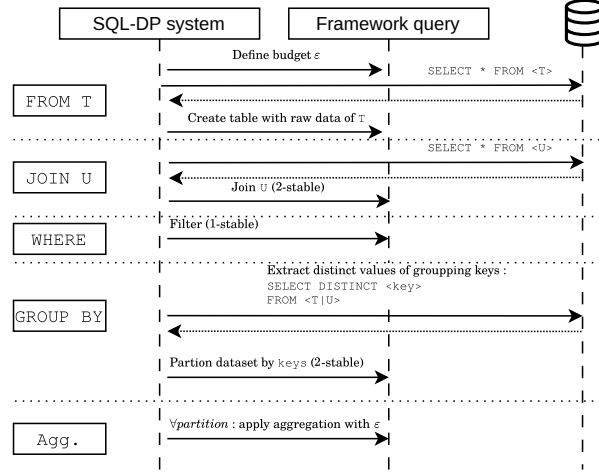


Fig. 1: Sequence diagram of a DP-SQL system using a framework to process an SQL query

5 Experiments

In this section, we present a qualitative and quantitative analysis of six DP-SQL-based systems: proxy-based ones (Qrlew [?] and Chorus [?]), integrated ones (Smartnoise [?] and DOP-SQL [?]) and framework-based ones (PINQ [?] and Tumult [?]). These systems are examined in terms of their ability to be used to protect data in a DBMS. We adopt the perspective of an analyst who would like to query a dataset without being an expert in DP.

5.1 Experimental Setup

Experiments were conducted using a Jupyter notebook with Python 3.11.2 on a Linux Debian 12 distribution. The machine is equipped with an INTEL CORE i9-13950HX processor, featuring 32 cores with a maximum turbo frequency of 5.50 GHz, and 64 GB of RAM.

Database DP-SQL systems are evaluated using the synthetic TPC-H benchmark dataset [?], a standard reference within the database community for modeling complex relational schema. The dataset represents a global supply chain environment consisting of 8 interconnected tables, centered on core entities such as Customer, Order, Lineitem, Part, and Supplier. This schema provides a diverse range of data types, including continuous numerical values (quantities, prices), categorical attributes, and unstructured text. Our evaluation utilizes a scale factor of 1, resulting in a total volume of approximately 8×10^6 tuples.

Tested SQL Construct The analysis of the constraints and challenges involved in using SQL for DP has led to the identification of five test parameters, depending on the type of SQL construct used:

- **Aggregation Functions:** Evaluation of COUNT, COUNT DISTINCT, AVG, SUM, and extremum functions (MIN/MAX).
- **Table Sensitivity Level:** The proximity of the queried table to the primary private data:
 - **Primary (PK):** Tables containing direct private data (e.g., Customer, Supplier).
 - **1 Foreign Key Reference (1FK):** Tables with a single foreign key to a private table (e.g., Orders).
 - **2 Foreign Key Reference (2FK):** Tables linked via multiple foreign keys (e.g., Lineitem).
 - **Public (PUB):** Non-sensitive reference tables. (e.g. nation)
- **Joins (J):** Presence of relational join operations.
- **Selection (S):** Use of WHERE clauses to filter records.
- **Histogram (H):** Whether the query produces a frequency distribution.

Quantitative Metrics All the queries presented in Section ?? are executed $NB = 25$ times each with a timeout of 60 seconds. For each query Q , the utility is measured by retrieving the protected values $P_{i,j}^Q$, where $i \in [0, NB]$ is the iteration number, and j is the bin if the query is a histogram. These values are compared with the unprotected results: U_j^Q through Mean Absolute Percentage Error (MAPE) :

$$MAPE_Q = \frac{100}{NB.n} \sum_{i=0}^{NB} \sum_{j=1}^n \frac{|P_{i,j}^Q - U_j^Q|}{U_j^Q}$$

To avoid excessively high MAPE values, which are linked to low values of U_j^Q , queries are selected so that they never return a result below the arbitrary value 1500 (except for COUNT_DISTINCT and Select_Isolate).

We also measure the execution time T'_Q compared with the time without system DP-SQL T_Q , as well as the slowdown factor: T'_Q/T_Q . These measures enable the creation of a utility table (Table ??), a deceleration table (Table ??) and a support table (Table ??).

Privacy Parameters: The systems are tested with a small budget ($\epsilon = 0.1$). We observed other values of ϵ ; the errors varied with the budget, but these differences did not provide any basis for comparison between the systems. We therefore chose a budget that we believed to be consistent with practices designed to protect privacy. δ is set to $\frac{1}{n^2} = 10^{-12}$ (where n is the size of the database), as recommended in [?].

Some systems have other free parameters. These are set in accordance with the operating instructions. We chose to keep systems with free parameters even though we know that introduces a bias in comparison. Due to space constraints, the results with other parameter values are not shown. The experiments can be repeated with different values using the source code.

5.2 Query Set

TPC-H provides benchmark queries, but these are designed to test the performance of DBMSs. They were not relevant to our test cases because most returned raw data (non-statistical queries, outside the scope of the data model) and others performed multiple aggregations within the same query (which falls outside the scope of support for all the systems under consideration).

We have therefore defined two sets of queries for unit and pair-wise testing (Appendix ?? provides details of the query sets).

Unit Test: This set aims at verifying whether the systems are capable of meeting the basic requirements of a DP-SQL system. It covers five areas: support for queries, table privacy, supported join types, histogram support, and aggregation functions. All the unit test queries, and the questions raised, are set out in Table ??.

Pairwise: A key aspect of using SQL is the composability of basic constructs to create more complex queries. This is a major point to investigate in order to understand how DP-SQL systems can be used in real-world scenarios. This set contains queries combining some constructs along the axes discussed in Section ?. To reduce the combinatorial explosion in the number of queries, a pair-wise approach is used: for each pair of parameters (aggregation, table privacy, join, selection, histogram), this set tests all possible discrete combinations of those parameters. Table ?? shows all the queries in the pair-wise set.

5.3 Results Tables

This section presents the results of the quantitative experiment, along with an analysis of the systems and their results. Tables ?? and ?? show, respectively, the error introduced by the systems and the slowdown factor. A numerical value indicates that a query is supported and executed successfully. The symbol '⊖' indicates that the execution time limit (timeout) has been exceeded, whilst '✗' indicates an execution error. The symbol '-' indicates that a query is considered unsupported for the purposes of the evaluation.

For unit queries, a query is marked with '-' if it is not supported by the mechanism's theoretical capabilities. For pairwise queries, support is defined relative to the unit results: a pairwise query is only evaluated if each of its unit components has been successfully validated (i.e. has produced a numerical metric). Thus, if at least one component is marked '-', '⊖' or '✗' in the unit evaluation, the pairwise query is also marked '-'. This convention allows the system's ability to combine validated unit functionalities to be evaluated exclusively, without confusing the limitations of the elementary components with those of their composition.

Furthermore, in Table ??, errors exceeding 100% are considered unusable in practice and are therefore shown in grey. The items in bold indicate the best system for each query (lowest error rate or slowdown factor).

Table ?? compares the utility of 'user-level' and 'tuple-level' systems even though the unit of privacy is different. It looks unfair for 'user-level' systems because, in theory,

more noise needs to be added to process a user’s entire dataset compared with a single tuple. However, in practice, this distinction is not clear-cut.

Using this data, it is possible to measure the systems’ throughput (Table ??). For each test suite, we note the total number of requests, the expected number, the percentage of successful requests, and the percentage of valid requests (successful requests with an average error rate of less than 100%). The items in bold indicate the systems with the best support, based on the test suites.

Table 1: Summary of the error in DP-SQL systems for $\epsilon = 0.1$.

Tests			User-level			Tuple-level		
			DOP-SQL	Tumult	Orlew	Chorus	SmartNoise	PINQ
Unit test	Select	Select_Isolate	2.00e+05	7.92e+02	2.10e+05	1.91e+03	X	1.15e+03
		Select_NonEqui	2.23e+02	1.65e-02	9.77e+00	2.33e-02	1.26e-02	1.32e-02
	Table Privacy	1FK	1.27e-01	5.25e-04	4.40e-01	1.09e-03	1.27e-01	8.46e-04
		2FK	3.64e-02	1.43e-04	1.58e+00	2.84e-04	-	1.67e-04
		FK+PUB	1.15e+01	9.95e-04	X	2.73e-03	X	1.74e-03
		PK	3.14e+00	6.16e-03	2.28e+00	1.21e-02	6.75e-03	5.74e-03
		PK_supplier	7.44e+04	7.28e-02	2.61e+01	1.22e-01	9.12e-02	7.34e-02
		PUB	9.72e-01	5.36e-03	0.00e+00	0.00e+00	6.42e-03	6.22e-03
	Join	JOIN_1to1	4.66e+01	1.59e-01	9.22e+01	9.23e+00	-	1.18e-01
		JOIN_Auto	1.30e+00	X	1.15e+01	X	-	X
		JOIN_Equi	8.35e+02	8.58e+00	9.69e+01	X	-	7.94e+00
		JOIN_FKs	1.23e-01	9.33e+01	3.51e-01	4.30e-01	-	9.33e+01
		JOIN_NonEqui	6.43e+03	8.26e+01	8.06e+02	-	-	-
	Histo.	Histo_NonSensitive	1.68e-01	3.34e-02	4.12e+01	1.41e-01	2.93e-02	6.37e-02
		Histo_PUB	7.56e-01	1.66e-01	1.43e+02	6.74e-01	1.69e-01	3.43e-01
	Aggregation	AVG	4.05e+01	1.55e-02	1.32e+01	-	1.14e-01	1.01e+02
		COUNT	1.38e+02	1.02e-02	1.12e+00	1.46e-02	6.45e-03	7.37e-03
		COUNT_DISTINCT	6.19e+05	6.00e+05	5.86e+05	5.93e+01	4.70e+01	1.02e+02
		MAX	X	4.16e+16	-	-	X	1.02e+07
		MIN	5.74e+05	3.80e+17	-	-	X	1.09e+00
		SUM	1.49e-01	1.60e-02	6.72e+00	-	1.76e-01	2.12e+01
Pairwise		COUNT_1FK_J_S_H	3.56e+00	○	X	1.24e-01	-	7.50e+01
		COUNT_2FK_J_S	2.60e+01	○	X	5.81e+00	-	9.92e+01
		COUNT_DIS_1FK_S	5.71e+04	5.79e+04	5.78e+04	1.58e+00	8.57e+01	X
		COUNT_DIS_2FK_J_H	9.41e+03	X	9.38e+03	1.72e+00	-	1.00e+02
		COUNT_DIS_PK_J	7.89e+03	4.76e-01	X	2.84e+01	-	9.99e+01
		COUNT_DIS_PUB_J_S_H	3.28e+04	2.08e+02	0.00e+00	0.00e+00	-	2.89e+02
		COUNT_PK_J_H	9.19e-01	X	1.10e+01	7.65e+00	-	9.35e+01
		COUNT_PUB_H	X	7.60e-01	0.00e+00	0.00e+00	7.41e-01	1.50e+00
		MAX_2FK_H	-	1.96e+15	-	-	-	1.23e+05
		MAX_PUB_H	-	2.22e+17	0.00e+00	-	X	5.11e+07
		MIN_1FK_S	X	3.74e+20	-	-	-	1.49e+06
		MIN_PK_J_S_H	X	X	-	-	-	1.17e+08
		AVG_1FK_H	X	○	3.16e+01	-	-	1.00e+02
		AVG_2FK_J	1.25e+01	○	7.54e+00	-	-	○
		AVG_PK_S	X	4.04e-01	1.24e+06	-	4.55e+00	1.01e+02
		AVG_PUB_S	1.36e+02	1.03e+00	0.00e+00	-	-	1.00e+02
		SUM_1FK	2.02e-03	9.34e+01	1.49e+00	-	-	9.34e+01
		SUM_2FK_J_S	2.22e+01	○	X	-	-	○
		SUM_PK_H	2.12e+01	5.09e+00	X	-	5.65e+02	2.20e+01
		SUM_PUB_H	X	5.03e+00	0.00e+00	-	4.91e+01	6.65e+01

Table 2: Summary of the slowdown factor in DP-SQL systems for $\epsilon = 0.1$

			User-level			Tuple-level		
			DOP-SQL	Tumult	Qrlew	Chorus	SmartNoise	PINQ
Unit test	Select	Select_Isolate	2.5e+1	5.2e+2	1.9e+2	5.2e+1	X	1.7e+2
		Select_NonEqui	1.9e+1	4.9e+2	1.6e+2	4.0e+1	2.8e+1	1.4e+2
	Table Privacy	1FK	4.2e+1	2.6e+2	1.2e+2	2.3e+1	3.0e+1	2.2e+2
		2FK	6.1e+1	1.1e+2	6.9e+1	8.6e+0	-	2.8e+2
		FK+PUB	3.78e+01	3.75e+02	X	3.16e+01	X	1.97e+02
		PK	2.5e+1	4.2e+2	1.7e+2	4.2e+1	2.8e+1	1.4e+2
		PK_supplier	1.8e+1	5.4e+2	1.8e+2	4.7e+1	3.0e+1	9.4e+1
		PUB	2.4e+1	3.8e+2	1.4e+2	3.8e+1	2.3e+1	1.4e+2
	Join	JOIN_1tol	2.3e+1	9.0e+2	1.5e+2	5.0e+1	X	5.4e+2
		JOIN_Auto	1.9e+1	X	1.1e+2	X	-	X
		JOIN_Equi	4.4e+0	3.2e+2	3.5e+1	X	-	1.4e+2
		JOIN_FKS	2.1e+1	3.2e+2	5.1e+1	1.6e+1	-	1.6e+2
		JOIN_NonEqui	7.8e+0	3.7e+2	7.9e+1	-	-	-
	Histo.	Histo_NonSensitive	2.9e+1	4.3e+2	1.8e+2	3.9e+1	3.2e+1	1.3e+2
		Histo_PUB	4.0e+1	4.6e+2	1.9e+2	4.2e+1	3.1e+1	1.4e+2
	Aggregation	AVG	1.4e+1	1.7e+3	1.7e+2	-	2.7e+1	1.4e+2
		COUNT	2.2e+1	3.7e+2	1.6e+2	3.8e+1	2.5e+1	1.2e+2
		COUNT_DISTINCT	1.4e+1	3.4e+2	1.2e+2	3.0e+1	2.1e+1	1.1e+2
		MAX	X	X	-	-	X	1.3e+2
		MIN	1.8e+1	7.9e+2	-	-	X	1.4e+2
		SUM	2.04e+01	1.20e+03	1.67e+02	-	2.79e+01	1.39e+02
Pairwise		COUNT_1FK_J_S_H	2.0e+1	○	X	1.3e+1	-	9.1e+1
		COUNT_2FK_J_S	6.1e+0	○	X	4.0e+0	-	5.9e+1
		COUNT_DIS_1FK_S	1.5e+1	4.6e+1	2.1e+1	5.0e+0	5.1e+0	X
		COUNT_DIS_2FK_J_H	1.9e+0	X	4.8e+0	1.1e+0	-	7.2e+0
		COUNT_DIS_PK_J	5.2e+0	6.8e+1	X	5.8e+0	-	3.0e+1
		COUNT_DIS_PUB_J_S_H	2.1e+1	7.6e+2	2.1e+2	5.4e+1	-	2.2e+2
		COUNT_PK_J_H	3.1e+1	X	5.0e+1	1.1e+1	-	1.2e+2
		COUNT_PUB_H	X	3.6e+2	1.3e+2	3.5e+1	2.3e+1	1.3e+2
		MAX_2FK_H	-	1.6e+2	-	-	-	1.5e+2
		MAX_PUB_H	-	6.7e+2	1.2e+2	-	X	1.3e+2
		MIN_1FK_S	X	3.1e+2	-	-	-	2.0e+2
		MIN_PK_J_S_H	X	X	-	-	-	1.3e+0
		AVG_1FK_H	X	○	9.1e+1	-	-	1.0e+2
		AVG_2FK_J	2.1e+0	○	1.4e+1	-	-	○
		AVG_PK_S	X	4.8e+2	1.3e+2	-	2.3e+1	1.1e+2
		AVG_PUB_S	1.4e+1	5.2e+2	1.4e+2	-	-	1.4e+2
		SUM_1FK	3.4e+1	1.6e+2	7.9e+1	-	-	1.4e+2
		SUM_2FK_J_S	5.1e+0	X	X	-	-	○
		SUM_PK_H	4.5e+1	5.0e+2	X	-	3.3e+1	1.0e+2
		SUM_PUB_H	X	3.4e+2	1.2e+2	-	2.3e+1	1.3e+2

Table 3: Support of SQL queries by DP-SQL systems for $\epsilon = 0.1$

	Unit						Pairwise					
			Passed		Useful				Passed		Useful	
	Nb	Exp.	Nb	Rate	Nb	Rate	Nb	Exp.	Nb	Rate	Nb	Rate
DOP-SQL	21	21	20	95.2%	12	57.1%	20	18	12	66.7%	7	38.9%
Tumult	21	21	20	95.2%	16	76.2%	20	16	12	75.0%	7	43.8%
Qrlew	21	19	18	94.7%	14	73.7%	20	17	12	70.6%	9	52.9%
Chorus	21	16	14	87.5%	13	81.3%	20	8	8	100.0%	8	100.0%
SmartNoise	21	15	11	73.3%	11	73.3%	20	6	5	83.3%	4	66.7%
PINQ	21	20	19	95.0%	15	75.0%	20	18	17	94.4%	8	44.4%

5.4 Results Analysis

This section analyses the results presented for each system, with a brief explanation of each system’s internal workings and how they shape its behavior.

Chorus implements several differentially private mechanisms; the experiments reported here evaluate its primary algorithm, FLEX [?]. This algorithm quantifies the impact of joins on query stability by relying on the *max frequency* (mf), defined as the maximum number of occurrences of the most frequent value of an attribute. In practice, Chorus requires these values to be statically declared in a schema file, whereas the original Flex framework computed them dynamically. This restriction causes two failures in our test suite: the JOIN_Equi and JOIN_Auto queries, whose join predicate operates on continuous-domain attributes (`customer.acctbal`, `orders.totalprice`) for which no mf can be defined a priori. On the pairwise workload, Chorus successfully handles all expected queries (100% support), with a low relative error, albeit slightly higher than that observed in unit tests. Chorus thus offers sound compositional support for SQL constructs, at the cost of a restricted query coverage imposed by the requirement for static metadata.

Qrlew is based on the systematic rewriting of all aggregations as weighted sums, a strategy that yields satisfactory coverage on our test suite (73.7% on unit queries, 52.9% on pairwise queries) but structurally excludes MAX and MIN operators, which are not reducible to this scheme. Furthermore, this coverage rate is tempered by the presence of bugs and occasional errors that further degrade the effective pass rate.

DOP-SQL dynamically selects the most appropriate DP mechanism for each query, which should in theory guarantee full coverage of our test suite. However, our experiments reveal the sporadic introduction of disproportionately large absolute errors on straightforward queries (COUNT, PK, PK_supplier, etc.), affecting 1 to 3% of iterations per query. These outliers appear to stem from the system, as anomalous values were observed in the version of the code provided by the authors of DOP-SQL. We have not been able to determine the cause of these errors. These values are not constant; a query may be correct most of the time, with an acceptable average error rate, and produce an anomalous value only rarely. This phenomenon introduces structural instability: beyond occasional non-responses, a produced result carries a non-negligible probability of being entirely incorrect. This explains DOP-SQL’s low useful support rate (57.1% on unit queries, 38.9% on pairwise queries), despite its adaptive mechanism selection strategy.

SmartNoise SQL is an OpenDP-based tool [?] designed for the production of differentially private statistics over tabular data, exposed through a SQL interface. Its primary structural limitation is the single-table assumption: it is not possible to simultaneously query multiple tables containing private data, which precludes queries involving two foreign keys (2FK) or joins between private tables. Access to reference tables through a single foreign key (1FK) remains supported. This constraint is fundamentally at odds with the relational model and severely restricts query coverage, both for unit and composed queries. SmartNoise SQL therefore addresses a specific use case — the production of aggregate statistics over a single tabular source re-exposed via SQL — and does not constitute a general-purpose solution to the relational DP-SQL problem. In our experiments, the protection level was set to tuple-level rather than user-level, consistent

with the standard assumption of a relational schema in which individual granularity is encoded in the data model. This configuration yields acceptable performance within this restricted scope (73.3% on unit queries, 66.7% on pairwise queries).

Tumult is a differentially private framework built on OpenDP [?], designed to track the sensitivity and stability of transformations applied to tabular data while automatically managing the privacy budget. It supports all aggregation functions in our test suite, multiple protection levels (with the exception of public table optimization), as well as joins, histograms, and selections, making it suitable for implementing a complete DP-SQL system. Its primary operational overhead stems from loading entire tables into local memory prior to processing. This behavior severely penalizes queries over large tables — in particular `lineitem` (6,000,000 tuples at scale factor 1) — and causes execution timeouts in our experiments.

PINQ is a differentially private framework based on the notion of c -stability, which enables query composition as a sequence of transformations over tabular data while bounding their cumulative impact on global stability. As a consequence of this constraint, PINQ natively supports only 1-to-1 joins; for joins with higher cardinality, it truncates the result to the first matching tuple in order to enforce the 1-to-1 property. This behavior structurally limits result fidelity on queries involving multi-valued joins.

6 Discussion

Our qualitative and quantitative evaluation demonstrates that no existing DP-SQL system provides consistently useful responses across all queries: the uniform deployment of a DP-SQL system to protect statistical outputs from a relational database remains an experimental endeavor.

Two opposing design strategies have emerged to address this challenge. The first consists in specializing — and thereby restricting — the supported SQL fragment, while ensuring that useful results are produced in every case within that scope. Chorus (Flex), which is limited to counting queries, and SmartNoise SQL, which does not support joins, are representative of this approach. The second strategy aims to maximize SQL coverage, accepting a broader range of query constructs. DOP-SQL achieves this by dynamically selecting and optimizing the DP mechanism based on the query at hand, while Qrlew adopts a uniform rewriting strategy that reduces all aggregations to sum operations.

With respect to result support and accuracy, specialized systems constitute a viable alternative when operational requirements fall within their supported scope. Chorus in particular stands out as the most stable system: although it fails on queries involving joins over attributes for which no max frequency can be statically defined, it responds with high stability on all queries it is designed to support. For this system, inspecting the query prior to execution is generally sufficient to determine whether it will be handled successfully. By contrast, systems in the second family offer theoretically broader coverage, but at the cost of stability — manifesting as bugs and execution timeouts. Consequently, while the absolute number of useful query responses is comparable to that of Chorus and SmartNoise SQL, the ratio of useful to expected responses is lower. Although these systems can be deployed without deep knowledge of differential pri-

vacy, it is difficult to anticipate whether a given query will produce a result at all, let alone a useful one.

For a more complete comparison with the broader landscape of DP data protection, one can employ a generic DP framework after manually implementing an SQL interpreter in a hosted language, as illustrated in Section ??.

This methodology, however, introduces a significant additional barrier relative to the two families described above: correctly implementing query interpreters in these frameworks requires a solid understanding of the theoretical foundations of differential privacy. Qualitatively, the support rates achieved by our Tumult and PINQ implementations are of the same order of magnitude as those of the other systems, with the exception of Chorus on the pairwise workload. Furthermore, these implementations exhibit notable instability: between one quarter and one half of pairwise queries either fail or produce unusable results without any prior indication (a behavior analogous to that observed for Qrlew and DOP-SQL). An additional limitation of framework-based approaches is the requirement to process data in local memory: both Tumult and PINQ must retrieve entire table contents before performing any computation, which substantially increases query execution time. Framework-based approaches thus appear well-suited to handling specific, well-defined use cases, but cannot serve as general-purpose SQL interpreters to be plugged into an existing relational database management system.

7 Conclusion & Future Work

This evaluation shows that no existing DP-SQL system offers consistently useful answers across the full range of relational queries: deploying differential privacy uniformly over a relational database remains an experimental endeavor. Two opposing strategies emerge, each with a distinct failure mode: specialized systems restrict the supported SQL fragment but remain highly predictable within it, while general-purpose approaches (DOP-SQL, Qrlew) or approaches implemented atop a generic DP framework (Tumult, PINQ) extend coverage at the cost of stability, additional expertise requirements, or undetected privacy leaks. Overall, the three approaches are well suited to specific, well-defined use cases, but currently no DP-SQL system exists as a general-purpose, plug-and-play SQL interpreter for relational DBMS.

In future work, we would like to propose more refined tests that enable the identification of a wider range of SQL constructs and related queries. In particular, by incorporating other commonly used datasets, we aim to replicate experiments carried out on different systems and generalise the findings. Furthermore, it is possible to develop a tool that addresses these limitations by incorporating a modular approach, thereby enabling specific tools such as FLEX to be adapted for broader coverage.

A Unit Test Query Set

Tables ?? shows the queries for both test sets. For the unit test queries, Table ?? shows each query, along with an associated question that the query covers. For the pairwise set, Table ?? shows the queries composed of different values for the five test axes.

Name	Description
Select_Isolate	The selection isolates a single person in the dataset
Select_NonEqui	Inequality operators for selection?
1FK	Tables that reference a private table
2FK	Tables that reference two different private tables
Fk+PUB	Table with two foreign keys, one private and the other public (partsupp)
PK	Directly private table (customer)
PK_supplier	Can a system support a second private table (supplier), and how does the number of tuples affect the results?
PUB	Optimization for public tables (publishing results without noise)
JOIN_1to1	Joins where each tuple is joined to at most one other tuple.
JOIN_Auto	Self-joins (requires handling table renaming)
JOIN_Equi	General equijoin
JOIN_FKs	Equijoins in the specific case of a FK-PK reference
JOIN_NonEqui	Join using an inequality operator
Histo_NonSensitive	Histograms using non-sensitive attributes as keys (e.g., mktsegment for customer)
Histo_PUB	Histograms using public attributes as keys (e.g., nationkey for customer)
AVG	Average aggregation
COUNT	Count aggregation
COUNT_DISTINCT	Count distinct attribute
MAX	Max aggregation
MIN	Min aggregation
SUM	Sum aggregation

(a) Unit tests queries

Aggregation	Table	J	S	H
COUNT	PK	J		H
	2FK	J	S	
	1FK	J	S	H
	PUB			H
COUNT DISTINCT	PK	J		
	2FK	J		H
	1FK	J	S	
	PUB	J	S	H
AVG	PK		S	
	2FK	J		
	1FK			H
	PUB		S	
SUM	PK			H
	2FK	J	S	
	1FK			
	PUB			H
MIN MAX	PK	J	S	H
	2FK			H
	1FK		S	
	PUB	J		H

(b) Pair-wise queries

Table 4: Set of queries for testing DP-SQL systems